

MetaTrader 5 / Статистика и анализ

Вероятностный метод главных компонент на MQL5

0

Evgeniy Chernish

74

Введение

Анализ главных компонент (PCA) — один из фундаментальных методов машинного обучения и анализа данных. Он широко применяется для уменьшения размерности, извлечения признаков, сжатия информации и визуализации многомерных пространств.

Традиционно PCA рассматривают как геометрический метод: данные проецируются на подпространство меньшей размерности так, чтобы сохранить максимум дисперсии. Наряду с этим существует и вероятностная формулировка метода (Probabilistic PCA), в которой та же задача записывается в виде модели с латентными переменными.

Именно вероятностный подход раскрывает дополнительные возможности PCA. Он позволяет оценивать параметры через EM-алгоритм. Также можно применять информационные критерии и кросс-валидацию для выбора числа главных компонент, а также вычислять функцию правдоподобия и оценивать соответствие новых данных обученной модели. Кроме того, PPCA служит концептуальным фундаментом для более сложных вероятностных моделей, таких как факторный анализ и анализ независимых компонент (ICA). Освоение вероятностной трактовки PCA существенно упрощает понимание этих моделей.

В данной статье мы пройдем путь от теории к практике:

- Рассмотрим математику классического и вероятностного PCA,
- Реализуем универсальный класс модели на MQL5, объединяющий оба подхода,
- Проверим корректность работы класса и оценим его вычислительную эффективность путем сравнения с эталонной реализацией из библиотеки Scikit-Learn,
- Исследуем проблему выбора числа главных компонент — от классического анализа "каменистой осыпи" (Scree Plot) до вероятностных методов (информационные критерии AIC/BIC и кросс-валидация),
- Рассмотрим применение показателя правдоподобия (Score) для обнаружения смены распределения данных на участках Out-of-Sample.

Классический компонентный анализ

Идея метода главных компонент заключается в понижении размерности данных с минимальной потерей информации. Представим набор точек на плоскости (два признака). Обычно они образуют вытянутое облако: вдоль одного направления разброс большой, а вдоль другого — заметно меньше. PCA находит направление наибольшего разброса и строит на нём новую ось (главную компоненту). Затем каждая исходная точка данных переносится на эту ось (проецируется). В результате вместо двух исходных координат у наблюдения остаётся одна — её положение на новой оси.

На рис. 1 показан простой пример:

- красные точки — исходные двумерные данные,
- синяя пунктирная линия — главная ось (подпространство),
- синие точки — ортогональные проекции исходных данных на эту ось, где координата каждой точки вдоль оси задает значение главной компоненты,
- чёрный крестик — центр данных (вектор средних μ).

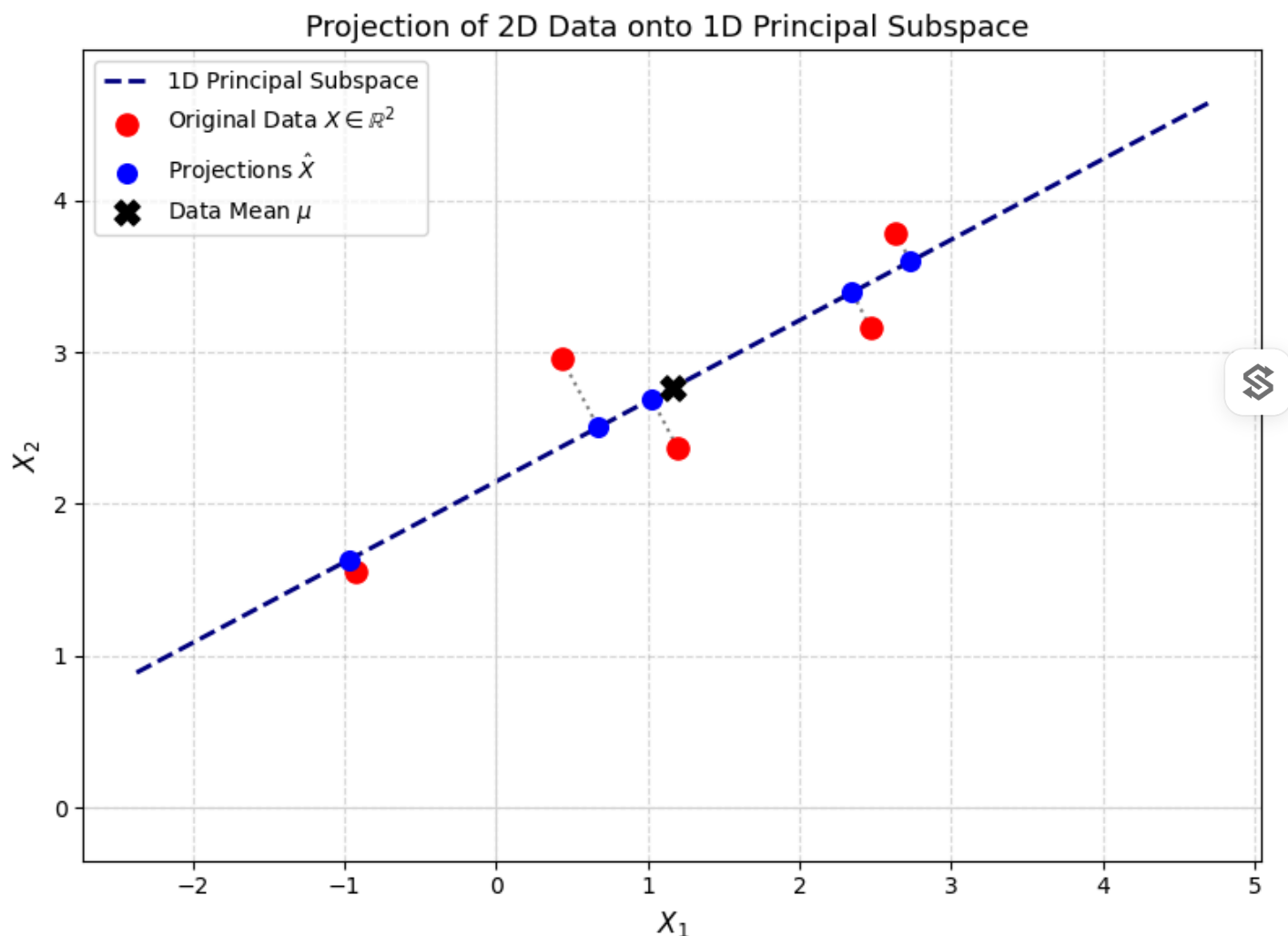


Рис. 1. Проекция двумерных данных на одномерное главное подпространство

Математическая формулировка

Пусть у нас есть матрица данных X размера $N \times D$, где N — число наблюдений, а D — число признаков. Для дальнейшего анализа данные X необходимо центрировать: вычисляем вектор средних μ , после чего формируется центрированная матрица X_{cent} .

Сразу рассмотрим самый простой случай — проекцию на одномерное подпространство ($M=1$). Для этого нужно найти единичный вектор направления $u_1 \in \mathbb{R}^D$, вдоль которого разброс данных будет максимальным. Скалярная проекция центрированного наблюдения ($x_n - \mu$) на этот вектор даёт значение первой главной компоненты:

$$z_{n1} = (\mathbf{x}_n - \boldsymbol{\mu})^T \mathbf{u}_1$$

Дисперсия полученных проекций z_{n1} вдоль направления u_1 определяется выражением:

$$\mathbf{u}_1^T \mathbf{S} \mathbf{u}_1$$

где S — выборочная ковариационная матрица данных размера $D \times D$.

Чтобы найти направление u_1 , вдоль которого дисперсия проекций максимальна, необходимо решить задачу оптимизации с ограничением на единичную длину вектора. Её решение напрямую связано со свойствами ковариационной матрицы S : направление наибольшей дисперсии совпадает с собственным вектором этой матрицы, отвечающим её наибольшему собственному значению λ_1 . Этот вектор называют первым главным направлением (Principal Direction / Axis).

Одномерный случай легко обобщается на произвольную размерность $M > 1$. Для этого берут M собственных векторов, соответствующих M наибольшим собственным значениям, и формируют из них ортонормированный базис $U_M = [u_1, u_2, \dots, u_M]$, размера $D \times M$. Проекция центрированных данных на этот ортонормированный базис определяется как:

$$Z_{pca} = X_{cent} U_M$$

Полученная матрица Z_{pca} ($N \times M$) содержит главные компоненты.

По сути, классический PCA сводится к трем последовательным операциям:

1. Оценить среднее μ и ковариационную матрицу S ,
2. Найти M собственных векторов матрицы S , соответствующих наибольшим собственным значениям,
3. Спроецировать центрированные данные на полученный базис.



Важно различать: собственный вектор u_1 задает направление оси в пространстве признаков, а главная компонента z_{n1} — это координата проекции n -го наблюдения на эту ось.

Вероятностный метод PCA

В отличие от классического PCA, основанного на геометрической проекции, вероятностный PCA формулирует задачу понижения размерности через модель с латентными (скрытыми) переменными. Такой подход дает ряд существенных преимуществ:

- позволяет использовать EM-алгоритм, который эффективен, когда нужны только M главных компонент без явного построения полной ковариационной матрицы S ,
- дает явную функцию правдоподобия, позволяющую сравнивать PPCA с другими моделями и корректно подбирать размерность главных компонент,
- позволяет генерировать новые данные из выученного распределения.

Вероятностный PCA — это линейно-гауссовская модель, в которой все маргинальные и условные распределения являются нормальными. Это значительно упрощает как математические выкладки, так и реализацию.

Математическая модель

Введём явную латентную переменную $\mathbf{z} \in \mathbb{R}^M$, соответствующую координатам в главном подпространстве. По предположению модели эта переменная имеет многомерное нормальное распределение с нулевым математическим ожиданием и единичной ковариацией:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \mathbf{0}, I)$$

Условное распределение наблюдаемой переменной $\mathbf{x} \in \mathbb{R}^D$ при заданном значении латентной переменной $\mathbf{z}$ также является нормальным:

$$p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x} \mid W\mathbf{z} + \boldsymbol{\mu}, \sigma^2 I)$$

где,

- W — матрица весов размера $D \times M$, столбцы которой образуют главное подпространство,
- μ — вектор среднего,
- σ^2 — дисперсия шума.

Интегрируя по латентным переменным z , получаем маргинальное распределение наблюдаемых данных, которое также является гауссовским:

$$p(\mathbf{x}) = \int p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} = \mathcal{N}(\mathbf{x} | \mu, C)$$

где, ковариационная матрица C определяется как:

$$C = WW^T + \sigma^2 I$$

Модельная ковариационная матрица C отличается от выборочной ковариационной матрицы данных S . Они совпадают только в предельном случае без сжатия размерности ($M=D$).

Таким образом, маргинальное распределение $p(\mathbf{x})$ полностью определяется параметрами μ , W и σ^2 .

По правилу Байеса апостериорное распределение латентной переменной $p(\mathbf{z}|\mathbf{x})$ при заданном наблюдении $\mathbf{x}$ выражается как:

$$p(\mathbf{z} | \mathbf{x}) = \mathcal{N}(\mathbf{z} | M^{-1}W^T(\mathbf{x} - \mu), \sigma^2 M^{-1})$$

где матрица M определяется как:

$$M = W^T W + \sigma^2 I_M$$

Оценка параметров методом максимального правдоподобия (аналитическое решение)

Вероятностную модель PCA удобно представить в виде ориентированного графа (рис.2), который наглядно показывает зависимости между переменными и параметрами.

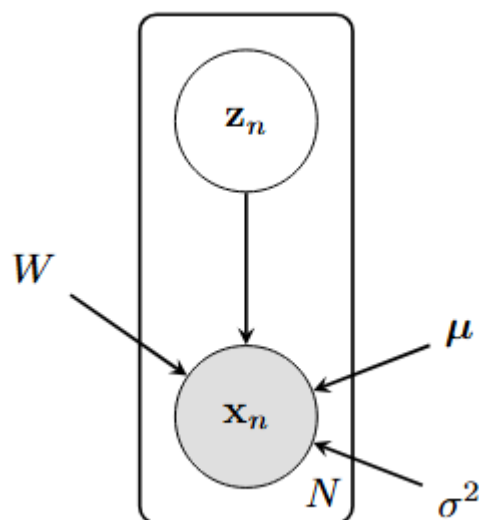


Рис. 2. Ориентированный граф вероятностной модели PCA

На графической схеме:

- закрашенный круг обозначает наблюдаемую переменную x — вектор признаков,
- белый круг обозначает латентную переменную z — координаты наблюдения в скрытом пространстве,
- рамка с индексом N указывает, что все наблюдения независимы и одинаково распределены (i.i.d.),
- стрелка от z_n к x_n означает, что каждое наблюдение порождается своей скрытой переменной,
- параметры модели (μ, W, σ^2) — константы, которые необходимо оценить по данным.

Логарифмическая функция правдоподобия выборки определяется как:

$$\ln p(X | \mu, W, \sigma^2) = -\frac{N}{2} [D \ln(2\pi) + \ln |C| + \text{Tr}(C^{-1}S)]$$

Приравнивая частную производную по μ к нулю, получаем оценку максимального правдоподобия для среднего:

$$\mu_{\text{MLE}} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$$



Параметры W и σ^2 имеют точное аналитическое решение, которое выражается через спектральное разложение выборочной ковариационной матрицы S :

$$W_{\text{MLE}} = U_M (L_M - \sigma^2 I_M)^{1/2} R$$

$$\sigma_{\text{MLE}}^2 = \frac{1}{D - M} \sum_{i=M+1}^D \lambda_i$$

где,

- U_M — матрица $D \times M$, столбцы которой являются ортонормированными собственными векторами выборочной ковариационной матрицы S ,
- L_M — диагональная матрица ($M \times M$) составленная из M наибольших собственных значений λ ,
- R — произвольная ортогональная матрица поворота ($M \times M$), (обычно принимают $R=I$),
- σ^2 — средняя дисперсия по отброшенным направлениям (необъяснённая дисперсия).

Таким образом, мы можем полностью рассчитать все параметры вероятностной модели, используя стандартные алгоритмы классического PCA. Достаточно найти собственные векторы и значения ковариационной матрицы S , после чего матрица весов W и дисперсия шума σ^2 вычисляются по приведенным выше формулам.

Единственное различие заключается в выборе матрицы поворота R : при аналитическом решении полагают $R=I$, что делает столбцы W строго ортогональными. Если параметры находятся итеративно через EM-алгоритм, матрица R оказывается произвольной — столбцы W формируют то же самое подпространство, но ортогональность между ними не гарантируется.

ЕМ-алгоритм для модели PPCA

Хотя для PPCA существует точное аналитическое решение, оценки параметров можно находить и с помощью EM-алгоритма. На первый взгляд это может показаться избыточным, но в пространствах с высокой размерностью с вычислительной точки зрения бывает выгоднее использовать итерационную процедуру EM-алгоритма, а не работать непосредственно с выборочной ковариационной матрицей.

Кроме того, этот же подход можно также распространить на модель факторного анализа (обобщение вероятностного PCA), для которой не существует решения в замкнутой форме.

Вывод ЕМ-алгоритма следует стандартной схеме:

- Записываем логарифмическую функцию правдоподобия полных данных (наблюдаемые + латентные переменные),
- Вычисляем её математическое ожидание по апостериорному распределению латентных переменных (при старых значениях параметров) – получаем целевую функцию Q,
- Максимизируем функцию Q по параметрам и получаем новые оценки.

Поскольку в вероятностной модели PCA наблюдения предполагаются независимыми, логарифмическая функция правдоподобия при полных данных принимает вид:

$$\ln p(X, Z | \mu, W, \sigma^2) = \sum_{n=1}^N \{\ln p(\mathbf{x}_n | \mathbf{z}_n) + \ln p(\mathbf{z}_n)\}$$

Вычисляя математическое ожидание логарифмической функции правдоподобия при полных данных относительно апостериорного распределения по латентным переменным, получим целевую функцию Q:

$$\begin{aligned} Q = \mathbb{E} [\ln p(\mathbf{X}, \mathbf{Z} | \mu, W, \sigma^2)] = & - \left[\frac{NM}{2} \ln(2\pi) \right. \\ & + \frac{1}{2} \text{Tr} \left(\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^T] \right) \\ & + \frac{ND}{2} \ln(2\pi\sigma^2) \\ & + \frac{1}{2\sigma^2} \sum_{n=1}^N \|\mathbf{x}_n - \mu\|^2 \\ & - \frac{1}{\sigma^2} \sum_{n=1}^N (\mathbf{x}_n - \mu)^T W \mathbb{E}[\mathbf{z}_n] \\ & \left. + \frac{1}{2\sigma^2} \text{Tr} \left(\left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^T] \right] W^T W \right) \right] \end{aligned}$$

Е-шаг

Это математическое ожидание зависит от апостериорного распределения только через достаточную статистику нормального распределения, вычисляемую по формулам:

$$\mathbf{E}[\mathbf{z}_n] = M^{-1} W^T (\mathbf{x}_n - \mu)$$

$$\mathbf{E}[\mathbf{z}_n \mathbf{z}_n^T] = \sigma^2 M^{-1} + \mathbf{E}[\mathbf{z}_n] \mathbf{E}[\mathbf{z}_n]^T$$

М-шаг

На М-шаге выполняем максимизацию по W и σ^2 , сохраняя апостериорную достаточную статистику фиксированной:

$$\sigma_{\text{new}}^2 = \frac{1}{ND} \sum_{n=1}^N \{ \|\mathbf{x}_n - \boldsymbol{\mu}\|^2 - 2\mathbb{E}[\mathbf{z}_n]^T W^T (\mathbf{x}_n - \boldsymbol{\mu}) + \text{Tr}(\mathbb{E}[\mathbf{z}_n \mathbf{z}_n^T] W^T W) \}$$

$$W_{\text{new}} = \left[\sum_{n=1}^N (\mathbf{x}_n - \boldsymbol{\mu}) \mathbb{E}[\mathbf{z}_n]^T \right] \left[\sum_{n=1}^N \mathbb{E}[\mathbf{z}_n \mathbf{z}_n^T] \right]^{-1}$$

ЕМ-алгоритм выполняется путем инициализации параметров, а затем поочередного вычисления достаточной статистики апостериорного распределения в латентном пространстве на Е-шаге и уточнения значений параметров с помощью формул на М-шаге.



Практическая реализация: класс PPCA

Вся математика вероятностного метода главных компонент реализована в классе CPPCA.

```
//+-----+
//|                                     PPCA.mqh |
//|                                     Eugene |
//|                                     https://www.mql5.com |
//+-----+
#property copyright "Eugene"
#property link      "https://www.mql5.com"

#include <Math\Stat\Normal.mqh>

enum ENUM_PPCA_SOLVER
{
    EM,           // EM
    eig,          // Eig
    svd           // SVD
};

//+-----+
//| Класс Probabilistic PCA (PPCA) |
//+-----+
class CPPCA
{
private:
    int          m_N;           // Количество наблюдений
    int          m_D;           // Исходная размерность
    int          m_M;           // Целевая скрытая размерность
    int          m_seed;        // Seed для генератора случайных чисел

    vector       m_mu;          // Вектор средних
    matrix       m_Xcent;       // Центрированные данные (N x D)
    double       m_Xnorm2;      // Квадрат нормы Фробениуса

    matrix       m_W;           // Матрица весов (D x M)
    double       m_sigma2;      // Дисперсия шума
```



```

matrix      m_E_zn;          // Апостериорные средние  $E[z_n]$  ( $M \times N$ )
matrix      m_sum_E_znzn;    // Сумма вторых моментов  $\text{Sum}_n(E[z_n * z_n^T])$  ( $M \times M$ )
matrix      m_X_E;          // Матрица  $X_{\text{cent}}^T @ E_{zn}^T$  [ $D \times M$ ]
matrix      m_P_M;          // Матрица проекции  $M^{-1} * W^T$  [ $M \times D$ ]
matrix      m_transform;     // Классическая ортогональная проекция PCA ( $N \times M$ )
matrix      m_U_m;          // Собственные векторы главных компонент  $U_m$  ( $D \times M$ )
double      m_Qhist[];      // Массив истории целевой функции  $Q$ 

void         InitializeW(const matrix &W_init);    // инициализация весов  $W$ 
void         InitializeSigma2(double sigma2_init); // инициализация дисперсии  $\sigma^2$ 

bool         FitEM(const matrix &W_init, const double sigma2_init, const int max_iter);
bool         FitEig(void);
bool         FitSVD(void);
void         ComputeExpectations(void); // E-step
double       UpdateSigma2();
matrix       UpdateW();
void         UpdateParameters(void); // M-step
double       EMStep(void);
double       Q(void); // expected value of the complete data log-likelihood

public:

vector       m_explained_variance; // Объясненная дисперсия первых  $M$  компонент
vector       m_explained_variance_ratio_; // Доля объясненной дисперсии первых  $M$  компонент

CPPPCA(const int M = 2, const int seed = -1)
:
    m_M(M),
    m_seed(seed),
    m_N(0),
    m_D(0),
    m_sigma2(1.0),
    m_Xnorm2(0.0)
{
    if(m_seed != -1)
        MathSrand((uint)m_seed);
    else
        MathSrand(GetTickCount());
}

~CPPPCA(void) {}

//--- Основной метод обучения
bool Fit(const matrix &X,
        const ENUM_PPCA_SOLVER solver = EM,
        const int max_iter = 30,
        const double tol = 1e-3);

//--- Перегрузка Fit с передачей начальных параметров для EM
bool Fit(const matrix &X,
        const matrix &W_init,
        const double sigma2_init = 0.0,
        const ENUM_PPCA_SOLVER solver = EM,
        const int max_iter = 30,
        const double tol = 1e-3);

matrix Transform(const matrix &X_new);

```



```

matrix          Project(const matrix &X_new);
//--- Обратное преобразование из сжатого пространства Z [N x M] в X_rec [N x D]
bool            InverseTransform(const matrix &Z, matrix &X_rec) const;
bool            InverseProjected(const matrix &Z, matrix &X_rec) const;
//--- Оценка правдоподобия
bool            ScoreSamples(const matrix &X, vector &ll) const; // правдоподобие
double          Score(const matrix &X) const; // среднее правдоподобие
//--- Информационные критерии и суммарный Log-Likelihood
double          TotalLogLikelihood(const matrix &X) const;
double          AIC(const matrix &X) const;
double          BIC(const matrix &X) const;
//--- Восстановленная ковариационная матрица
matrix          GetCovariance(void) const; // Матрица C = W*W^T +  $\sigma^2$ I
//--- Геттеры
matrix          GetW(void) const          { return m_W; }
double          GetSigma2(void) const     { return m_sigma2; } // noise_variance_
matrix          GetTransform(void) const  { return m_transform; }
matrix          GetProjected(void) const  { return m_E_zn.Transpose(); }
vector          GetMu(void) const         { return m_mu; }
matrix          GetXcent(void) const      { return m_Xcent; }
void            GetQhist(double &qhist[]) const { ArrayCopy(qhist, m_Qhist); }
};

```

Конструктор класса принимает два параметра:

- M — целевая размерность скрытого пространства,
- seed — зерно генератора псевдослучайных чисел. Если seed = -1 (по умолчанию), происходит случайная инициализация параметров в EM-алгоритме. Для других способов обучения этот параметр не нужен.

Метод Fit имеет две перегрузки: со случайной инициализацией параметров и с возможностью задать начальные значения матрицы весов W и дисперсии шума σ^2 (актуально для EM-алгоритма).

```

//+-----+
//| Перегрузка Fit с пользовательской инициализацией W и sigma2 |
//+-----+
bool CPPCA::Fit(const matrix &X,
               const matrix &W_init,
               const double sigma2_init = 0.0,
               const ENUM_PPCA_SOLVER solver = EM,
               const int max_iter = 30,
               const double tol = 1e-3)
{
    m_N = (int)X.Rows();
    m_D = (int)X.Cols();

    if(m_N == 0 || m_D == 0 || m_M <= 0 || m_M > m_D)
    {
        Print(__FUNCTION__, ": Ошибка размерностей! N=", m_N, ", D=", m_D, ", M=", m_M);
        return false;
    }

    //--- центрирование данных
    m_mu = X.Mean(0);
    m_Xcent.Init(m_N, m_D);
    for(int n = 0; n < m_N; n++)
        m_Xcent.Row(X.Row(n) - m_mu, n);
}

```

```

//--- метод обучения
if(solver == eig)
{
    return FitEig();
}
else
if(solver == svd)
{
    return FitSVD();
}

return FitEM(W_init, sigma2_init, max_iter, tol);

}

//+-----+
//| Основной метод обучения |
//+-----+
bool CPPCA::Fit(const matrix &X,
               const ENUM_PPCA_SOLVER solver = EM,
               const int max_iter = 30,
               const double tol = 1e-3)
{
    matrix empty_W(0, 0);
    return Fit(X, empty_W, 0.0, solver, max_iter, tol);
}

```

Основные входные параметры:

- X — матрица исходных данных,
- solver — выбранный алгоритм обучения (EM, eig или svd),
- max_iter и tol — параметры остановки для EM.

Перед вычислением параметров метод Fit проводит проверку корректности входной матрицы, вычисляет вектор средних μ и формирует центрированную матрицу X_{cent} , после чего передаёт управление выбранному алгоритму обучения.

Аналитические методы: FitEig и FitSVD

Методы FitEig и FitSVD дают точное решение задачи PPCA по принципу "2 в 1": они выполняют классическое снижение размерности (PCA) и одновременно находят параметры вероятностной модели (W и σ^2). Для повышения производительности матричные операции внутри методов используют оптимизированные процедуры библиотеки OpenBLAS.

FitEig — спектральное разложение ковариационной матрицы

Предназначен для случая, когда $D \leq N$. Для вычисления матрицы S используется метод BlasL3SyRK. Это существенно быстрее стандартного матричного умножения:

```
matrix S = m_Xcent.Transpose() @ m_Xcent / (m_N - 1);
```

и избавляет от необходимости дополнительной симметризации матрицы S :

```
S = 0.5 * (S + S.Transpose())
```

перед подачей в метод EigenSymmetricDC, что также увеличивает время расчетов.

Вычисление собственных значений и векторов производится методом EigenSymmetricDC на базе алгоритма LAPACK Divide and Conquer (syevd). Поскольку метод возвращает собственные значения по

возрастанию, вектор L и столбцы матрицы U разворачиваются по убыванию с помощью функций FlipVector и FlipColumns.

К выделенным M столбцам применяется процедура коррекции знаков SvdFlip, обеспечивающая строгую детерминированность результатов и совпадение с реализацией scikit-learn.

На основе отброшенного спектра вычисляется остаточный шум σ_2 , после чего формируется матрица весов W. Далее рассчитываются классическая проекция PCA и матрица математического ожидания скрытых переменных E[z|x].

```
//+-----+
//| Спектральное разложение ковариационной матрицы S (Eig) |
//+-----+
bool CPPCA::FitEig(void)
{
//--- 0. проверка размерностей
    if(m_D > m_N)
    {
        PrintFormat("%s: ОШИБКА! Размерность D (%d) превышает число наблюдений N (%d)
                    \"Используйте метод SVD или EM!\", __FUNCTION__, m_D, m_N);
        return false;
    }

//--- 1. Ковариационная матрица S через вызов SYRK (A^T @ A)
    matrix S(m_D, m_D);
    double alpha = 1.0 / (m_N - 1.0);
    double beta = 0.0;
    if(!m_Xcent.BlasL3SyRK(BLASTRANS-T, alpha, beta, S, S))
    {
        Print(__FUNCTION__, ": Ошибка вычисления ковариационной матрицы через BlasL3SyRK"
        return false;
    }

//--- 2. Спектральное разложение симметричной матрицы через LAPACK (Divide & Conquer)
    matrix U; // матрица собственных векторов
    vector L; // вектор собственных значений
    ResetLastError();
    if(!S.EigenSymmetricDC(EIGVALUES-V, L, U))
    {
        int sys_err = GetLastError();
        int lapack_info = (int)MQLInfoInteger(MQL_LAST_OPENBLAS_ERROR);

        PrintFormat("%s: Ошибка EigenSymmetricDC (MQL Error: %d | LAPACK info: %d)",
                    __FUNCTION__, sys_err, lapack_info);
        return false;
    }

//--- 3. Сортировка собственных значений и векторов по убыванию
    FlipVector(L);
    FlipColumns(U);

//--- 3.1 ВЫЧИСЛЯЕМ explained_variance И explained_variance_ratio_
    m_explained_variance = L;
    m_explained_variance.Resize(m_M);
    double total_variance = L.Sum();
    if(total_variance > 0.0)
        m_explained_variance_ratio_ = m_explained_variance / total_variance;
```

```
//--- 4. Дисперсия шума sigma2
if(m_D > m_M)
{
    double sum_tail = 0.0;
    for(int i = m_M; i < m_D; i++)
        sum_tail += L[i];
    m_sigma2 = sum_tail / (m_D - m_M);
}
else
{
    m_sigma2 = 0.0; // Если берутся все компоненты, остаточный шум равен 0
}

//--- 5. Отбор первых M главных компонент
matrix U_m = U;
U_m.Resize(m_D, m_M);

//--- 5.1. Коррекция знаков U_m
SvdFlip(U_m);

//--- 5.2. Вычисление проекций m_transform (N x M)
m_Xcent.BlasL3GeMM(BLASTRANS_N, BLASTRANS_N, 1.0, U_m, 0.0, m_transform);

//--- 6. Матрица весов W = U_M * diag(sqrt(L_M - sigma2))
vector L_m = L;
L_m.Resize(m_M);

vector diff = L_m - m_sigma2;
diff.Clip(0.0, DBL_MAX); // Защита от отрицательной величины
vector scale = MathSqrt(diff);

matrix Lm_sqrt;
Lm_sqrt.Diag(scale);
m_W = U_m @ Lm_sqrt;

//--- 7. Вычисление математического ожидания скрытых переменных E[z|x]
vector scale_E = scale / L_m;
matrix L_inv_scale;
L_inv_scale.Diag(scale_E);
m_E_zn = (m_transform @ L_inv_scale).Transpose(); // m_E_zn (M x N)
m_P_M = L_inv_scale @ U_m.Transpose();
m_U_m = U_m;

return true;
}
```



FitSVD – сингулярное разложение центрированной матрицы данных

Метод SVD работает напрямую с центрированной матрицей данных. В реализации используется SingularValueDecompositionDC (LAPACK dgesdd, Divide & Conquer). Хотя в отдельных сценариях SingularValueDecompositionBisection оказывается быстрее, окончательный выбор был сделан в пользу DC из-за стабильности и производительности на больших матрицах. В результате сингулярного разложения транспонированные правые сингулярные векторы образуют ортонормированный базис главных компонент. В остальном метод повторяет вычисления FitEig.

```
//+-----+
//| Сингулярное разложение центрированной матрицы данных SVD |
```



```

//+-----+
bool CPPCA::FitSVD(void)
{
//--- 1. Вычисление SVD для центрированной матрицы m_Xcent (N x D)
vector S;          // Вектор сингулярных чисел (min(N, D))
matrix U_svd;      // Левые векторы U (N x min(N, D))
matrix VT;         // Правые векторы V^T (min(N, D) x D)

ResetLastError();
if(!m_Xcent.SingularValueDecompositionDC(SVDZ_S, S, U_svd, VT))
{
    int sys_err      = GetLastError();
    int lapack_info = (int)MQLInfoInteger(MQL_LAST_OPENBLAS_ERROR);

    PrintFormat("%s: Ошибка SVD_DC (MQL Error: %d | LAPACK info: %d)",
                __FUNCTION__, sys_err, lapack_info);
    return false;
}

//--- 2. Пересчет сингулярных чисел sigma_i в собственные значения lambda_i
vector L = (S * S) / (m_N - 1.0); // lambda_i = sigma_i^2 / (N - 1)
ulong num_avail_L = L.Size(); // Фактическое число вычисленных собственных значений

//--- 2.1 Вычисляем explained_variance и explained_variance_ratio_
m_explained_variance = L;
m_explained_variance.Resize(m_M);

double total_variance = L.Sum();
if(total_variance > 0.0)
    m_explained_variance_ratio_ = m_explained_variance / total_variance;

//--- 3. Дисперсия шума sigma2
if(m_D > m_M)
{
    double sum_tail = 0.0;
    for(int i = m_M; i < (int)num_avail_L; i++)
        sum_tail += L[i];

    m_sigma2 = sum_tail / (double)(m_D - m_M);
}
else
{
    m_sigma2 = 0.0; // Если берутся все компоненты (M = D), остаточный шум равен 0
}

//--- 4. Отбор первых M главных компонент (столбцы правых векторов V)
matrix U_m = VT.Transpose();
U_m.Resize(m_D, m_M); // (D x M)

//--- 4.1. Коррекция знаков U_m
SvdFlip(U_m);

//--- 4.2. Вычисление проекций m_transform (N x M)
m_Xcent.BlasL3GeMM(BLASTRANS_N, BLASTRANS_N, 1.0, U_m, 0.0, m_transform);

//--- 5. Матрица весов W = U_M * diag(sqrt(L_M - sigma2))
vector L_m = L;

```



```

L_m.Resize(m_M);

vector diff = L_m - m_sigma2;
diff.Clip(0.0, DBL_MAX);
vector scale = MathSqrt(diff);

matrix Lm_sqrt;
Lm_sqrt.Diag(scale);
m_W = U_m @ Lm_sqrt;

//--- 6. Вычисление математического ожидания скрытых переменных  $E[z|x]$ 
vector scale_E = scale / L_m;
matrix L_inv_scale;
L_inv_scale.Diag(scale_E);
m_E_zn = (m_transform @ L_inv_scale).Transpose(); // m_E_zn [M x N]
m_P_M = L_inv_scale @ U_m.Transpose();
m_U_m = U_m;

return true;
}

```



FitEM

Метод реализует итерационный EM-алгоритм, описанный в теоретической части статьи. Перед началом итераций оценивается норма Фробениуса центрированной матрицы данных m_Xnorm2 , а параметры W и σ^2 инициализируются случайными или заданными пользователем значениями. В процессе итераций метод поочередно вызывает E-шаг и M-шаг и вычисляет целевую функцию Q . Сходимость контролируется по комбинированному критерию относительного и абсолютного изменения значения Q . При достижении заданной точности tol или исчерпании лимита итераций max_iter алгоритм завершает работу, сохраняя историю сходимости.

```

//+-----+
//| Обучение с помощью EM-алгоритма |
//+-----+
bool CPPCA::FitEM(const matrix &W_init,
                  const double sigma2_init,
                  const int max_iter,
                  const double tol)
{
//--- Вычисляем Xnorm2 (квадрат нормы Фробениуса)
double norm = m_Xcent.Norm(MATRIX_NORM_FROBENIUS);
m_Xnorm2 = norm * norm;
//--- Инициализируем W и sigma2
InitializeW(W_init);
InitializeSigma2(sigma2_init);

ArrayResize(m_Qhist, max_iter);
ComputeExpectations(); // Предварительный E-шаг для инициализации E_zn, E_znzn и по
double Qlast = Q();
int actual_iters = 0;
bool converged = false; // Флаг успешной сходимости

//--- Цикл EM-оптимизации
for(int iter = 0; iter < max_iter; iter++)
{
//--- Выполняем 1 шаг EM (ComputeExpectations -> UpdateParameters -> Expected val
double Q = EMStep();

```



```

m_Qhist[iter] = Q;
actual_iters = iter + 1;

//--- проверка условия остановки |Qcurr - Qlast| / |Qlast|
double diff = MathAbs(Q - Qlast);
double rel_diff = diff / (MathAbs(Qlast) + 1e-15);

if(rel_diff < tol || diff < tol)
{
    PrintFormat("EM converged at %d iterations. Final Q value: %.5f", iter + 1, Q);
    converged = true;
    break; // Критерий сходимости выполнен
}

Qlast = Q;
}

if(!converged)
{
    PrintFormat("EM did not converge within %d iterations. Final Q value: %.5f", max_
}

ArrayResize(m_Qhist, actual_iters);
return true;
}

```



```

//+-----+
//| E-шаг |
//+-----+
void CPPCA::ComputeExpectations(void)
{
//--- 1. M = W^T * W (размер M x M)
    matrix M;
    m_W.BlasL3GeMM(BLASTRANS_T, BLASTRANS_N, 1.0, m_W, 0.0, M);

//--- 1.1 Добавляем sigma2 * I_M
    M = M + matrix::Identity(m_M, m_M) * m_sigma2;

//--- 2. i_M = M^-1
    matrix i_M = M.Inv();

//--- 3. P_M = i_M * W^T -> размер (M x D)
    i_M.BlasL3GeMM(BLASTRANS_N, BLASTRANS_T, 1.0, m_W, 0.0, m_P_M);

//--- 3.1 m_E_zn = m_P_M * Xcent^T -> размер (M x N)
    m_P_M.BlasL3GeMM(BLASTRANS_N, BLASTRANS_T, 1.0, m_Xcent, 0.0, m_E_zn);

//--- 4. Ковариация Cov[z|x] = sigma2 * M_z^-1
    matrix cov_z = i_M * m_sigma2;

//--- 5. Суммарные вторые моменты Sum(E[z_n * z_n^T]) = m_E_zn @ m_E_zn^T + cov_z * N
    matrix E_zn_znT;
    m_E_zn.BlasL3GeMM(BLASTRANS_N, BLASTRANS_T, 1.0, m_E_zn, 0.0, E_zn_znT);
    m_sum_E_znzn = E_zn_znT + cov_z * m_N;
}

```





```
//+-----+
//| Обновление дисперсии шума sigma2 (M-step) |
//+-----+
double CPPCA::UpdateSigma2()
{
//--- 1. T2 = -2 * sum( (Xcent^T @ E_zn^T) * W )
    double T2 = -2.0 * (m_X_E * m_W).Sum();

//--- 2. T3 = sum( m_sum_E_znzn * (W^T @ W) )
    matrix WtW = m_W.Transpose() @ m_W;
    double T3 = (m_sum_E_znzn * WtW).Sum();

//--- 3. Возвращаем новое значение на основе текущего m_W
    return (m_Xnorm2 + T2 + T3) / (m_N * m_D);
}

//+-----+
//| Обновление матрицы весов W (M-step) |
//+-----+
matrix CPPCA::UpdateW()
{
    return m_X_E @ m_sum_E_znzn.Inv(); // W_new = X_E @ inv(m_sum_E_znzn) -> размер (D
}

//+-----+
//| M-шаг |
//+-----+
void CPPCA::UpdateParameters(void)
{
//--- 1. m_X_E = Xcent^T @ E_zn^T (D x M)
    m_Xcent.BlasL3GeMM(BLASTRANS_T, BLASTRANS_T, 1.0, m_E_zn, 0.0, m_X_E);
//--- 2. sigma2_new
    m_sigma2 = UpdateSigma2();
//--- 3. W_new
    m_W = UpdateW();
}
```



```
//+-----+
//| Вычисление полного логарифма правдоподобия Q (Complete-Data LL) |
//+-----+
double CPPCA::Q(void)
{
//--- Проверяем, заполнена ли m_X_E. Если нет - вычисляем
    if(m_X_E.Rows() != m_D || m_X_E.Cols() != m_M)
    {
        m_Xcent.BlasL3GeMM(BLASTRANS_T, BLASTRANS_T, 1.0, m_E_zn, 0.0, m_X_E);
    }

//--- 1. N * M * log(2 * pi) / 2
    double term1 = (double)m_N * (double)m_M * MathLog(2.0 * M_PI) / 2.0;

//--- 2. След суммы матриц E_znzn = Trace(m_sum_E_znzn) / 2.0
    double term2 = m_sum_E_znzn.Trace() / 2.0;

//--- 3. N * D * log(2 * pi * sigma2) / 2
    double term3 = (double)m_N * (double)m_D * MathLog(2.0 * M_PI * m_sigma2) / 2.0;
```

```

//--- 4. Xnorm2 / (2 * sigma2)
double term4 = m_Xnorm2 / (2.0 * m_sigma2);

//--- 5. -sum( (Xcent^T @ E_zn^T) * W ) / sigma2
// m_X_E = m_Xcent.Transpose() @ m_E_zn.Transpose(); // (D x M)
double term5 = -1 * ((m_X_E * m_W).Sum()) / m_sigma2;

//--- 6. sum( m_sum_E_znzn * (W^T @ W) ) / (2 * sigma2)
matrix WtW = m_W.Transpose() @ m_W; // (M x M)
double term6 = (m_sum_E_znzn * WtW).Sum() / (2.0 * m_sigma2);

//---- Итоговое значение Complete-Data Log-Likelihood
return -1 * (term1 + term2 + term3 + term4 + term5 + term6);
}

//+-----+
//| Выполнение одной итерации ЕМ-алгоритма |
//+-----+
double CPPCA::EMStep(void)
{
//--- 1. Е-шаг: пересчет апостериорных мат ожиданий E_zn и E_znzn
ComputeExpectations();
//--- 2. М-шаг: обновление параметров sigma2 и W
UpdateParameters();
//--- 3. Вычисление Q
return Q();
}

```



После завершения этапа обучения класс CPPCA позволяет проецировать новые данные, восстанавливать исходные признаки и выполнять вероятностную диагностику.

Проекция новых данных

Класс разграничивает два типа проекции:

- Transform(X) — классическая ортогональная проекция PCA. Этот метод полностью аналогичен поведению transform() в scikit-learn,
- Project(X) — вычисляет апостериорное математическое ожидание скрытых переменных E(z|x).

```

//+-----+
//| Проекция новых данных (классический PCA) |
//+-----+
matrix CPPCA::Transform(const matrix &X_new)
{
    ulong rows = X_new.Rows();
    matrix X_cent = X_new;

//--- 1. Центрируем по обученному среднему
    for(ulong i = 0; i < rows; i++)
        X_cent.Row(X_cent.Row(i) - m_mu, i);

//--- 2. Умножаем на собственные векторы
    return X_cent @ m_U_m;
}

//+-----+
//| Вероятностная проекция новых данных |

```



```
//+-----+
matrix CPPCA::Project(const matrix &X_new)
{
    ulong rows = X_new.Rows();
    if(rows == 0 || m_P_M.Rows() != (ulong)m_M)
    {
        Print(__FUNCTION__, ": Модель не обучена или пустые данные!");
        return matrix::Zeros(0, 0);
    }

    matrix X_cent = X_new;

    //-- 1. Центрируем по обученному среднему
    for(ulong i = 0; i < rows; i++)
        X_cent.Row(X_cent.Row(i) - m_mu, i);

    //-- 2.  $E[Z|X] = X\_cent @ P\_M^T [N \times D] @ [D \times M] \rightarrow [N \times M]$ 
    matrix Ez;
    X_cent.BlasL3GeMM(BLASTRANS_N, BLASTTRANS_T, 1.0, m_P_M, 0.0, Ez);

    return Ez;
}
```



Обратное преобразование данных

- InverseTransform(Z) — реконструирует данные по классическим ортогональным компонентам (аналог inverse_transform() в scikit-learn),
- InverseProjected(Z) — выполняет восстановление на основе апостериорных математических ожиданий $E[z|x]$

```
//+-----+
//|Обратное ортогональное преобразование из Z в X_rec (N x D) |
//+-----+
bool CPPCA::InverseTransform(const matrix &Z, matrix &X_rec) const
{
    ulong N = Z.Rows();
    ulong M = Z.Cols();
    ulong D = m_mu.Size();

    if(N < 1 || M < 1 || D < 1)
    {
        Print("Ошибка InverseTransform: некорректные размерности или модель не обучена");
        return false;
    }

    if(m_U_m.Rows() != D || m_U_m.Cols() != M)
    {
        Print(__FUNCTION__, ": Ошибка! Матрица U_m не сформирована или размерность M не с
        return false;
    }

    X_rec = Z @ m_U_m.Transpose(); //(N x D)

    //-- Добавляем математическое ожидание
    for(ulong n = 0; n < N; n++)
        X_rec.Row(X_rec.Row(n) + m_mu, n);
}
```



```

    return true;
}

//+-----+
//| Обратное вероятностное преобразование из  $E[Z|X]$  в  $X_{rec}$  (N x D) |
//+-----+
bool CPPCA::InverseProjected(const matrix &Z, matrix &X_rec) const
{
    ulong N = Z.Rows();
    ulong M = Z.Cols();
    ulong D = m_mu.Size();

    if(N < 1 || M < 1 || D < 1)
    {
        Print(__FUNCTION__, ": Ошибка: некорректные размерности или модель не обучена");
        return false;
    }

    if(m_W.Rows() != D || m_W.Cols() != M)
    {
        Print(__FUNCTION__, ": Ошибка! Несовпадение размерностей матрицы весов W");
        return false;
    }

    X_rec = Z @ m_W.Transpose(); //(N x D)

    // Добавляем математическое ожидание
    for(ulong n = 0; n < N; n++)
        X_rec.Row(X_rec.Row(n) + m_mu, n);

    return true;
}

```



Дополнительные методы

- `GetCovariance()`: Возвращает модельную ковариационную матрицу C ,
- `Score()`: Вычисляет среднее значение логарифмического правдоподобия,
- `AIC()` и `BIC()`: Возвращают значения информационных критериев Акаике и Байеса для оценки оптимальности выбранного количества компонент M ,
- `GetProjected()`: Возвращает матрицу апостериорных средних $E[Z|X]$ для обучающей выборки,
- `GetTransform()`: Возвращает стандартную ортогональную проекцию для обучающей выборки.

Сравнение результатов обучения с моделью Scikit-Learn

Для проверки корректности вычислений разработанного класса проведем сравнение с эталонной реализацией PCA из библиотеки `scikit-learn`.

Тестирование выполнялось на одном и том же датасете `Iris`:

- `Test_scikit_Iris.py` — обучает модель с использованием `scikit-learn.decomposition.PCA` и выводит результаты вычислений в журнал,
- `Test_PPCA_Iris.mq5` — загружает те же данные `Iris` (CSV-файлы `iris_X.csv` и `iris_y.csv`) и обучает модель `PPCA`. Дополнительно выводит для сравнения графики латентного пространства и классической ортогональной проекции, а также график сходимости ЕМ-алгоритма.

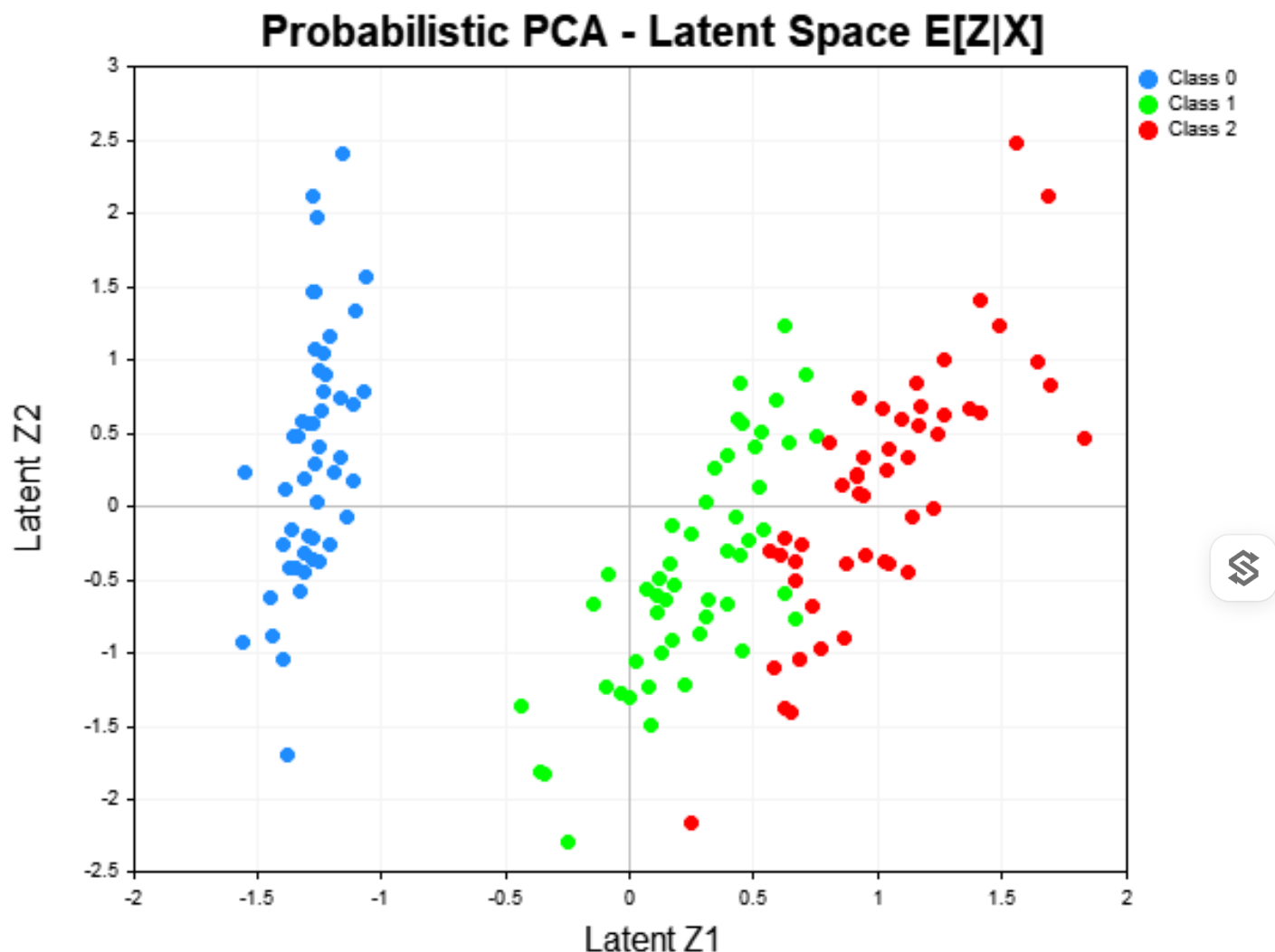


Рис. 3. Латентное пространство $E[Z|X]$ на данных Iris (SVD, $M=2$)

Основным объектом сравнения выбрана модельная ковариационная матрица C , которая в обеих реализациях возвращается методом `GetCovariance()`.

Результаты сравнения:

- Ковариационные матрицы C , полученные методами SVD и Eig в MQL5, совпадают с результатом scikit-learn с точностью до 6–8 знаков после запятой,
- Степень близости аналитических решений и EM-алгоритма удобно измерять через норму Фробениуса разности матриц. Матрица C , полученная с помощью EM, стремится к аналитическому решению по мере уменьшения порога `tol` или увеличения числа итераций `max_iter`.

Дополнительным подтверждением корректности реализации EM-алгоритма служит поведение целевой функции Q : она строго монотонно возрастает на каждой итерации (теоретическое свойство EM-алгоритма). История значений Q -функции показана на рис. 4.

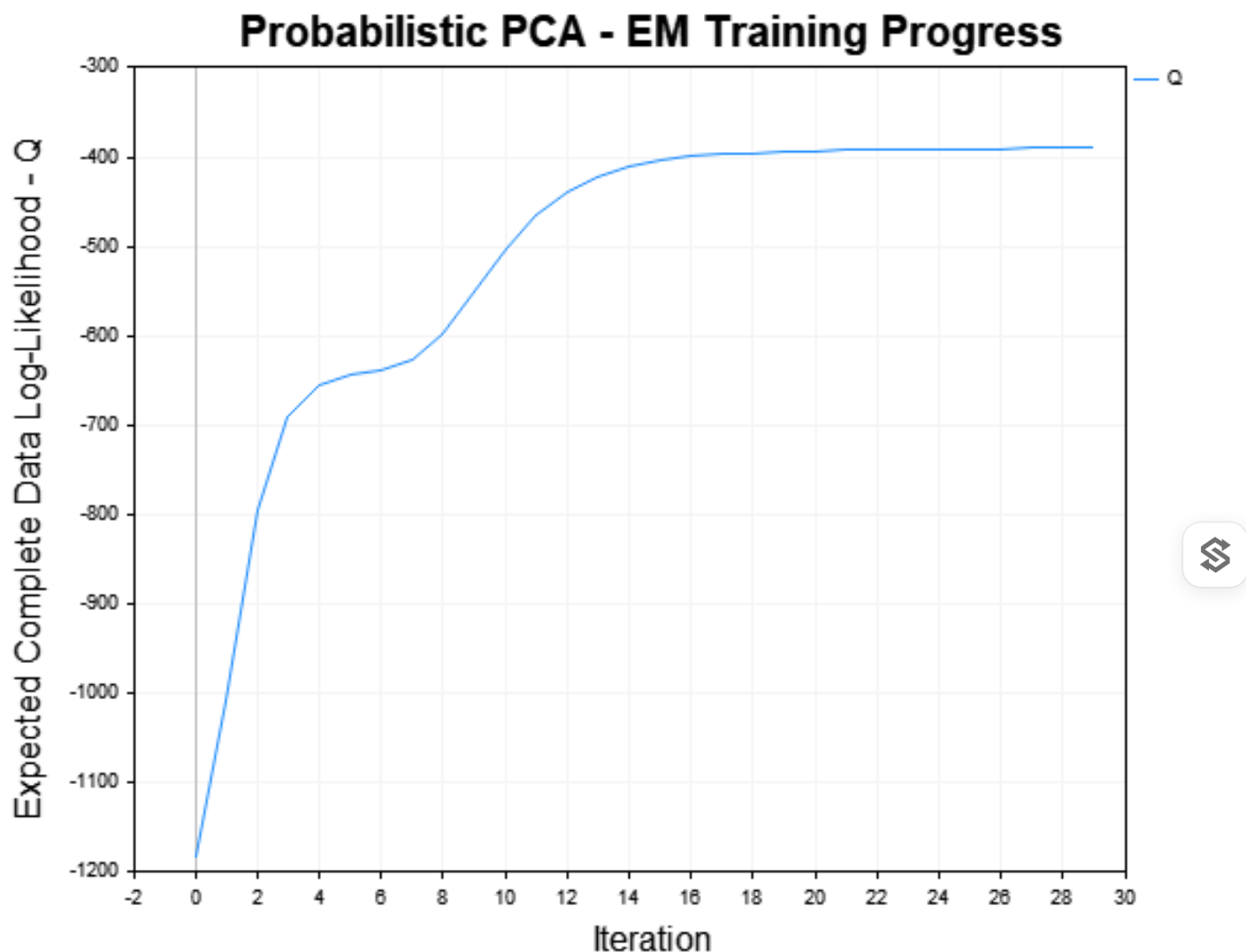


Рис. 4. Динамика Q-функции по итерациям EM-алгоритма на данных Iris

Оценка вычислительной эффективности

После проверки корректности перейдём к сравнению производительности реализации PPCA с библиотекой scikit-learn. Для этого подготовлено два скрипта:

- Benchmark_PPCA.mq5 — реализация PPCA (eig, svd и EM),
- Benchmark_scikit.py — модуль sklearn.decomposition.PCA (full, covariance_eigh, arpack и randomized).

Тестирование выполнялось на синтетических данных в двух противоположных сценариях при целевом числе главных компонент $M=10$ и истинном ранге матрицы $K=10$:

- Сценарий 1 (Большое число наблюдений): $N = 200000$ наблюдений, $D = 300$ признаков,
- Сценарий 2 (Высокая размерность): $N = 4000$, $D = 3000$.

Сценарий 1

Метод PPCA	Время обучения (сек)	Метод scikit	Время обучения (сек)
EIG	0.7820	covariance_eigh	0.4821
EM	0.9850	full_svd	5.8414
SVD	6.5940	arpack	1.2734
		randomized	1.9513

Метод EIG в MQL5 обучился всего за 0.782 с, показав скорость, сопоставимую с алгоритмом covariance_eigh из scikit-learn (0.482 с).

Сценарий 2

Метод PPCA	Время обучения(сек)	Метод scikit	Время обучения(сек)
EIG	4.6880	covariance_eigh	3.9609
EM	0.1880	full_svd	12.6904
SVD	13.3590	arpack	0.2372
		randomized	0.3452

При высокой размерности признаков EM-алгоритм в MQL5 продемонстрировал наилучший результат (0.188 с), даже немного опередив итерационные рандомизированные методы Python (arpack – 0.237 с, randomized – 0.345 с).

SVD-разложение в обоих языках оказалось самым медленным. Тем не менее на малых размерностях (в пределах 1000×1000), где разница во времени выполнения неощутима, именно SVD является наиболее надежным решением благодаря максимальной численной устойчивости – он работает с исходными данными напрямую, исключая накопление ошибок при построении ковариационной матрицы.



Выбор числа главных компонент: Scree Plot

Один из ключевых вопросов при использовании методов снижения размерности – определение оптимального количества скрытых компонент M. В классическом алгоритме PCA для этого традиционно используют критерий «каменистой осыпи» (Scree Plot). На рис.5 показан график каменистой осыпи (скрипт PCAScreePlot.mq5) для синтетических данных с истинным рангом матрицы = 3.

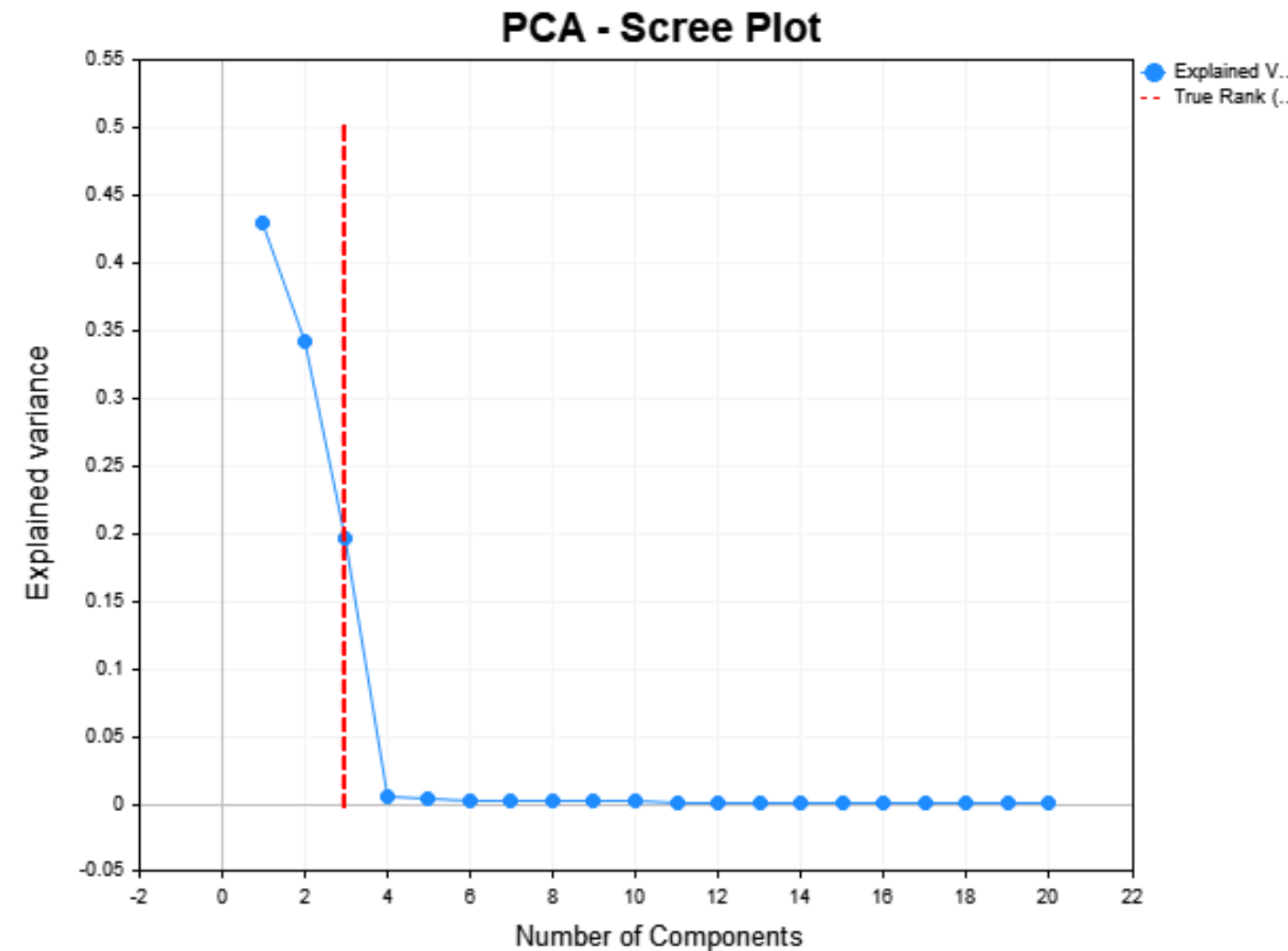


Рис. 5. График каменистой осыпи для синтетических данных, истинный ранг = 3

На графике хорошо виден так называемый "локоть" (излом): первые три компонента суммарно объясняют 96.8% дисперсии, а вклад остальных компонент резко падает (0.1–0.5 %). Для данного примера визуальный анализ Scree Plot позволил идеально определить истинный ранг. Однако на практике использовать Scree Plot оказывается проблематично. На реальных данных спектр собственных значений падает плавно, без выраженного излома. Выбор точки локтя или произвольного порога (например, удержание 90% дисперсии) становится эвристическим и зависит от субъективного решения исследователя. Это создает необходимость в более строгих, автоматизированных подходах к определению количества главных компонент.

В распоряжении вероятностного PCA оказываются более строгие инструменты для выбора числа главных компонент:

- кросс-валидация по логарифму правдоподобия (Log-Likelihood Cross-Validation),
- информационные критерии (AIC / BIC).

Выбор главных компонент через кросс-валидацию

Кросс-валидация позволяет выбрать такое число главных компонент M , при котором модель лучше всего объясняет новые данные. Реализация этого подхода представлена в скрипте CV_PPCA.mq5.



В ходе эксперимента на синтетическом наборе данных, содержащем 20 признаков и 3 главных компоненты, выполняется последовательный перебор предполагаемых размерностей в диапазоне от 1 до 19. Для каждого M проводится 5-кратная кросс-валидация: данные делятся на 5 блоков, модель обучается на четырёх из них, а на оставшемся тестовом блоке вычисляется средний логарифм правдоподобия методом Score().

По результатам строится кривая зависимости правдоподобия от числа компонент (рис. 6), что позволяет наглядно проследить динамику изменения качества модели в зависимости от выбираемой размерности:

- При $M < 3$ размерности скрытого пространства недостаточно для описания структуры данных, правдоподобие низкое,
- При $M = 3$ достигается глобальный максимум функции правдоподобия, что точно соответствует истинному числу скрытых компонент,
- При $M > 3$ модель начинает подстраиваться под случайный шум, из-за чего показатель правдоподобия начинает постепенно снижаться.

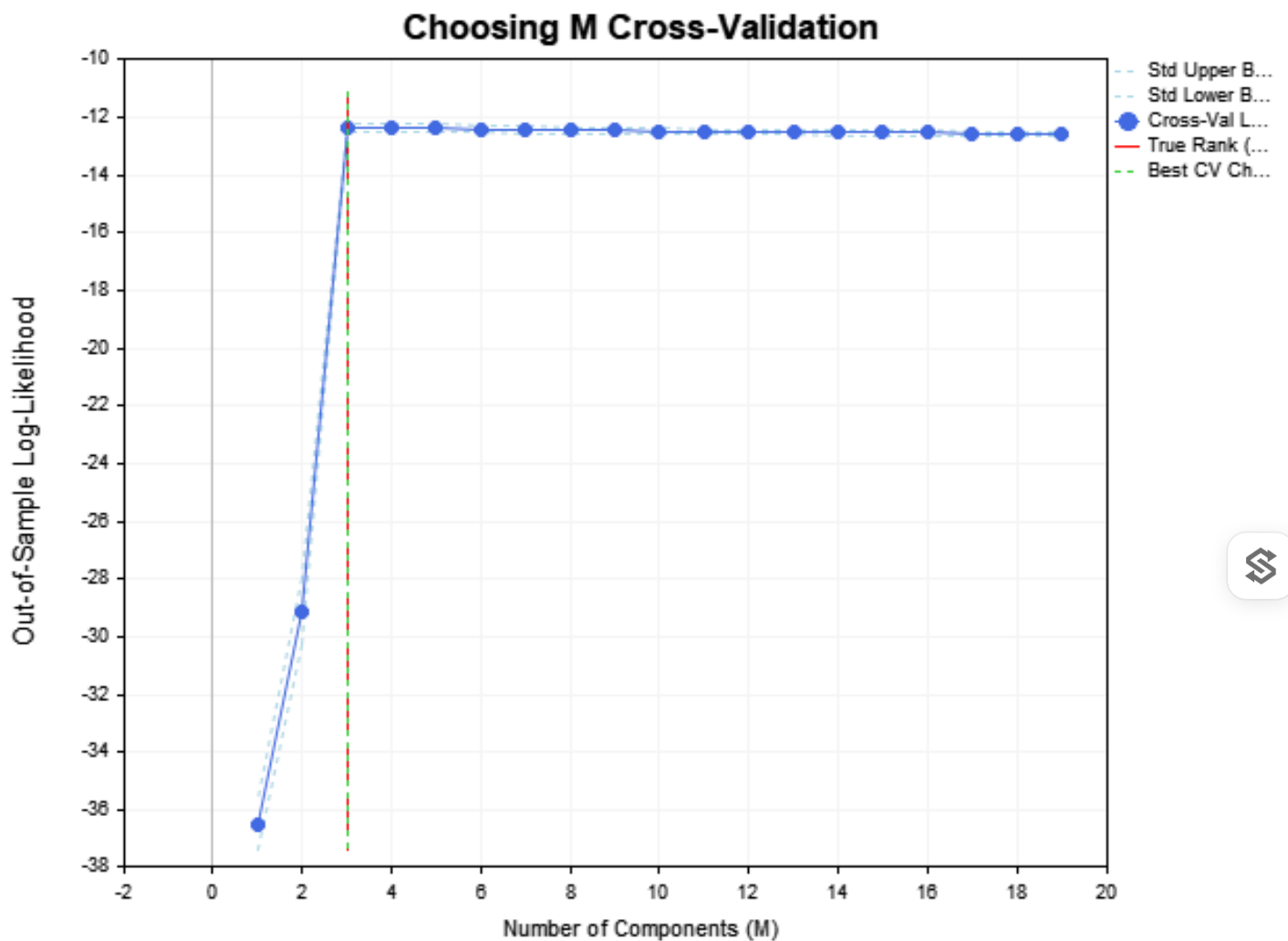


Рис. 6. Кривая правдоподобия на тестовых данных

Скрипт автоматически выбирает значение M, соответствующее максимуму этой кривой.

Выбор числа компонент через информационные критерии (AIC / BIC)

Хотя кросс-валидация дает точную оценку, она требует многократного переобучения модели, что вычислительно затратно на больших объемах данных. AIC и BIC — быстрая альтернатива кросс-валидации. Они оценивают модель на всей выборке и добавляют штраф за сложность.

Проверим этот подход на том же датасете, который мы использовали для кросс-валидации (скрипт AICBIC_PPCA.mq5). Напомним, что оценивать размерность M напрямую по логарифму правдоподобия на обучающей выборке нельзя. С ростом M правдоподобие монотонно возрастает, что неизбежно приводит к выбору максимально возможной размерности.

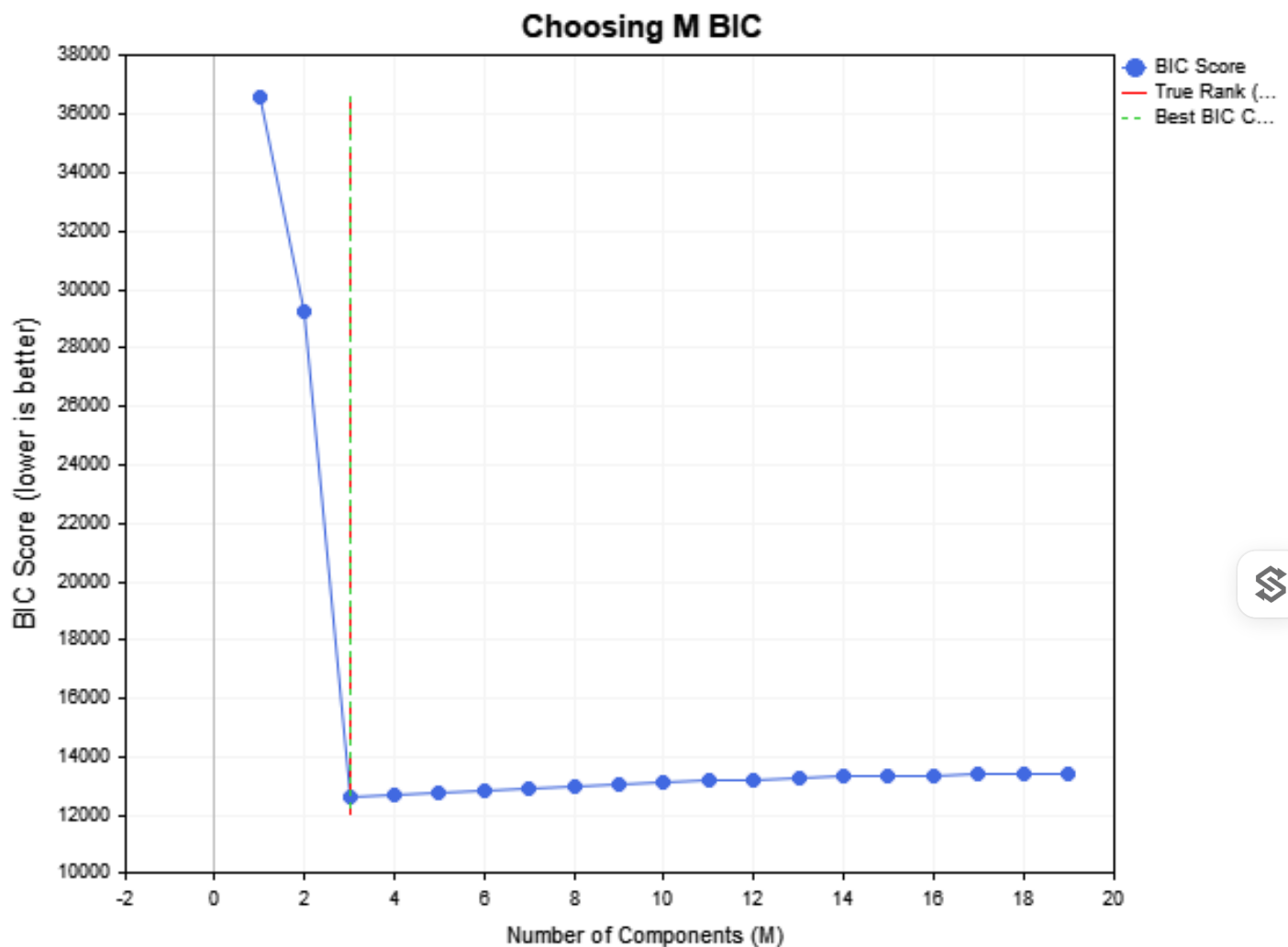


Рис. 7. Определение числа главных компонент с помощью BIC

Критерии AIC и BIC решают эту проблему, находя баланс между точностью аппроксимации и штрафом за усложнение архитектуры. Оптимальным считается значение M , при котором соответствующий критерий достигает минимума. Как видно из рис. 7, BIC успешно выбрал истинную размерность данных.

Обнаружение смены распределения данных

В финансовых временных рядах одна из главных проблем — нестационарность. Модель, обученная на одном историческом участке, теряет эффективность, когда распределение данных меняется.

Вероятностный PCA задаёт полноценную модель плотности. Благодаря этому появляется естественная метрика — средний логарифм правдоподобия (Score). Она показывает, насколько хорошо новые данные соответствуют структуре, выученной моделью ранее.

Учебный пример, демонстрирующий мониторинг смены распределения находится в скрипте `ScoreOOS.mq5`.

В данном примере мы генерируем синтетический набор данных в соответствии с порождающей моделью PPCA. Сначала создаются скрытые компоненты Z и матрица весов W , затем к ним добавляется шум. Количество скрытых компонент во всех режимах фиксировано и равно 2 (меняется только матрица весов или уровень шума).

Матрица данных X из 900 наблюдений и 5 признаков состоит из трёх последовательных режимов по 300 наблюдений:

- режим 0 (0...299): данные порождаются с матрицей весов W_0 и дисперсией шума $\sigma^2 = 0.2$,
- режим 1 (300...599): используется новая матрица весов W_1 при том же уровне шума. В результате кардинально меняется корреляционная структура признаков,

- режим 2 (600...899): возвращается исходная матрица весов W_0 , но дисперсия шума возрастает в 4 раза ($\sigma^2 = 0.8$).

Базовая модель P_0 обучается на первых 200 наблюдениях режима 0. Затем окно шириной 50 точек смещается по всему ряду с шагом 1 ($t=50...900$). На каждом шаге рассчитывается средний логарифм правдоподобия текущего окна относительно зафиксированной модели P_0 .

Поведение метрики Score можно увидеть на рис.8:

- в режиме 0 ($t < 300$) правдоподобие остаётся высоким и стабильным (Score = - 4) — это значит, что данные принадлежат тому же распределению,
- при переходе в Режим 1 метрика Score резко падает до значений около - 40. Модель фиксирует смену распределения: новые данные крайне маловероятны с точки зрения обученного распределения,
- в режиме 2 Score частично восстанавливается до уровня около -25. Модель узнаёт исходную структуру, но по-прежнему сигнализирует о смене распределения из-за возросшего шума.

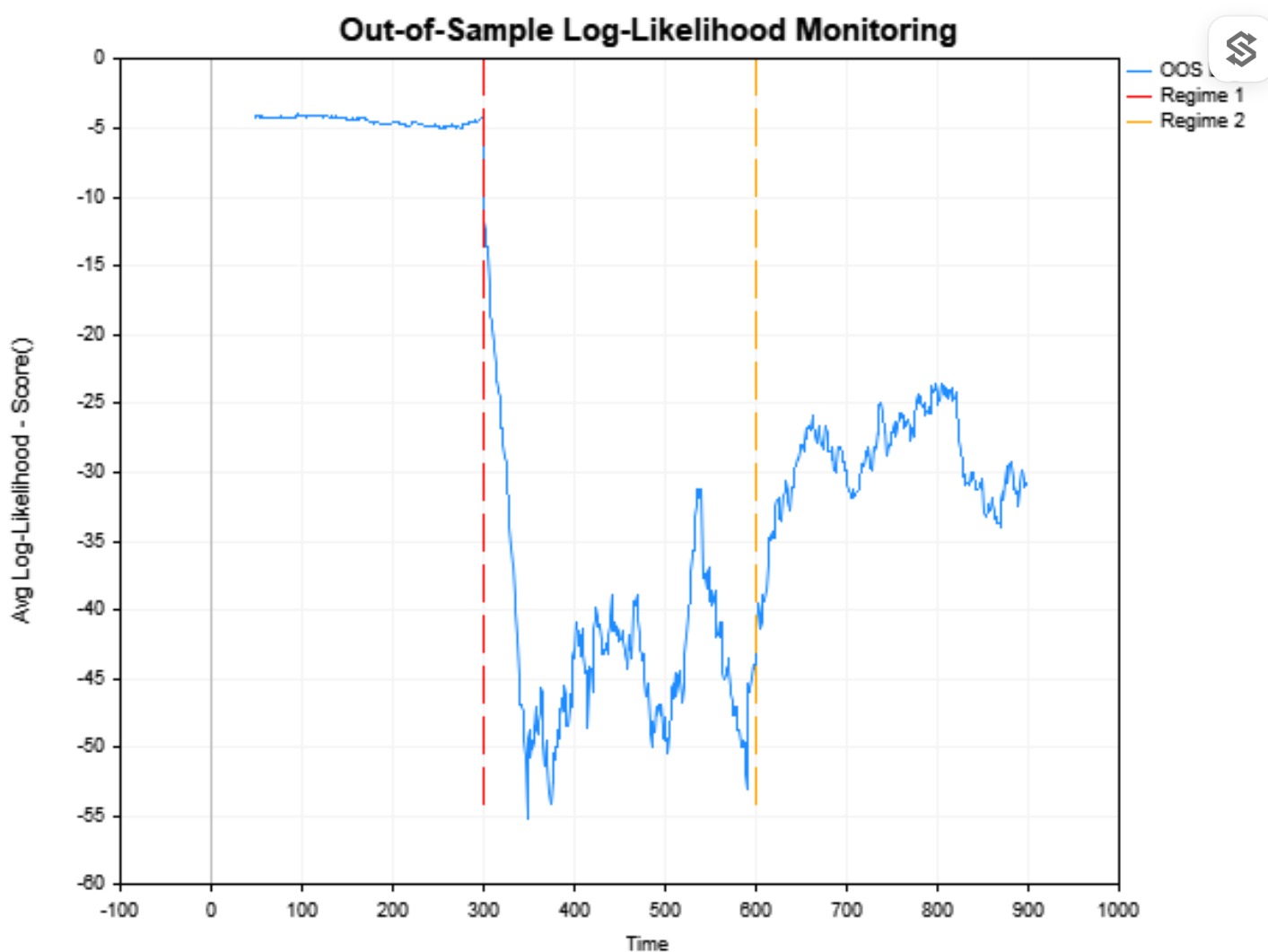


Рис. 8. Кривая среднего правдоподобия (Score), вычисленная в скользящем окне

Таким образом, метрика $\text{Score}()$ позволяет количественно оценивать, насколько новые данные соответствуют распределению, на котором модель была обучена. Резкое или устойчивое снижение правдоподобия указывает на изменение структуры данных.

При работе с финансовыми временными рядами эту метрику можно использовать для мониторинга состояния рынка. При выходе значения Score за допустимый порог торговая система может временно приостановить работу, а длительное падение правдоподобия — служить сигналом к обновлению параметров модели на более актуальном окне данных.

Чтобы запустить скрипт Benchmark_scikit.py и Test_scikit_Iris.py, вам понадобится интерпретатор Python и стандартный стек статистических пакетов. О том, как их установить, рассказывается в статье ["Python + MetaTrader 5: быстрый исследовательский контур для данных, признаков и прототипов"](#).

После установки пакетов поместите файлы Benchmark_scikit.py и Test_scikit_Iris.py в папку (например, Scripts/PPCA).

Запускайте скрипт через MetaEditor:

- в MetaEditor нажмите Файл → Открыть и выберите необходимый файл из папки PPCA,
- после открытия файла нажмите клавишу F7 (или кнопку "Компилировать" на панели инструментов),
- результаты вычислений должны появиться во вкладке Журнал.

Заключение

В статье мы последовательно рассмотрели два взгляда на метод главных компонент: классический геометрический PCA и его вероятностную формулировку (PPCA). Классический PCA находит ортогональное подпространство, максимизирующее дисперсию проекций. Вероятностный PCA представляет ту же задачу в виде модели с латентными переменными и позволяет оценивать параметры по принципу максимального правдоподобия. Благодаря этому появляются важные практические возможности: сравнение моделей через правдоподобие, использование информационных критериев и кросс-валидации для выбора числа главных компонент, а также генерация новых наблюдений.

Мы реализовали вероятностный метод главных компонент в виде класса CPPCA на MQL5 с тремя способами обучения – аналитическими (Eig, SVD) и итерационным (EM). Сравнение с моделью PCA из scikit-learn подтвердило корректность вычислений. В то же время реализация конкурентоспособна по скорости вычислений и позволяет гибко выбирать алгоритм в зависимости от соотношения числа наблюдений и признаков.

Отдельно были рассмотрены инструменты выбора размерности скрытого пространства (Scree Plot, Log-Likelihood CV, AIC/BIC) и пример использования метрики Score для обнаружения смены распределения на нестационарных данных. Последнее особенно актуально в задачах алгоритмической торговли, где важно вовремя заметить изменение рыночной структуры. Таким образом, вероятностный PCA не просто повторяет классический метод снижения размерности, а существенно расширяет его возможности, превращая геометрическую проекцию в полноценную вероятностную модель, удобную для анализа и принятия решений.

Программы, используемые в статье

#	Имя	Тип	Описание
1	PPCA.mqh	Включаемый файл	Модель вероятностного PCA
2	Test_scikit_Iris.py	Скрипт	Проверка корректности PPCA (scikit-learn, Python)
3	Test_PPCA_Iris.mq5	Скрипт	Проверка корректности PPCA (MQL5)
4	iris_X.csv	CSV	Признаки набора данных Iris
5	iris_y.csv	CSV	Метки набора данных Iris
6	PCAScreePlot.mq5	Скрипт	Выбор числа компонент по графику каменистой осыпи
7	CV_PPCA.mq5	Скрипт	Выбор числа компонент с помощью кросс-валидации
8	AICBIC_PPCA.mq5	Скрипт	Выбор числа компонент с помощью информационных критериев
9	Benchmark_PPCA.mq5	Скрипт	Проверка скорости работы модели PPCA
10	Benchmark_scikit.py	Скрипт	Проверка скорости работы модели scikit-learn

#	Имя	Тип	Описание
11	ScoreOOS.mq5	Скрипт	Пример обнаружения смены распределения с помощью метрики Score
12	MQL5.zip	Архив	Архив со всеми файлами статьи

Следующая статья >>

Evgeniy Chernish

[Украина](#)

[5551](#)



Другие статьи автора

- [ARD-регрессия на MQL5: автоматический отбор признаков и случайные признаки Фурье](#)
- [Реализация модели гауссовской смеси в MQL5](#)
- [Категориальная скрытая марковская модель на языке MQL5](#)
- [Основы байесовского вывода в дискретном и непрерывном случаях: от теории к практической реализации моделей](#)
- [Гипотеза случайности: поиск скрытых паттернов в ценовых рядах](#)
- [Выборочные методы марковских цепей Монте-Карло. Алгоритм HMC](#)

[Follow us](#) on socials for top articles and CodeBase updates